merge sort

```
#include <iostream>
#include <vector>
using namespace std;
void merge(vector<int>& arr, int low, int mid, int high) {
    vector<int> temp;
    int left = low;
    int right = mid + 1;
    while(left <= mid && right <= high) {
        if(arr[left] <= arr[right]) {
            temp.push_back(arr[left]);
            left++;
        }
        else {
            temp.push_back(arr[right]);
            right++;
        }
    }

    while(left <= mid) {
        temp.push_back(arr[left]);
        left++;
    }
    while(right <= high) {
        temp.push_back(arr[right]);
        right++;
    }
    for(int i = 0; i < temp.size(); i++) {
        arr[low + i] = temp[i];
    }
}
```

```
void mergeSort(vector<int>& arr, int low, int high) {
    if(low < high) {
        int mid = (low + high) / 2;
        mergeSort(arr, low, mid);
        mergeSort(arr, mid + 1, high);
        merge(arr, low, mid, high);
    }
}
int main() {
    int n;
    cin >> n;
    vector<int> arr(n);
    for(int i = 0; i < n; i++)
        cin >> arr[i];
    mergeSort(arr, 0, n - 1);
    for(auto x : arr)
        cout << x << " ";
    return 0;
}
#include <iostream>
#include <vector>
using namespace std;

int partition(vector<int>& arr, int low, int high) {

    int pivot = arr[low];

    int i = low;
    int j = high;

    while(i < j) {
```

```
        while(i < high && arr[i] <= pivot)
            i++;

        while(j > low && arr[j] > pivot)
            j--;

        if(i < j)
            swap(arr[i], arr[j]);
    }

    swap(arr[low], arr[j]);

    return j;
}

void quickSort(vector<int>& arr, int low, int high) {

    if(low >= high)
        return;

    int pivotIndex = partition(arr, low, high);

    quickSort(arr, low, pivotIndex - 1);

    quickSort(arr, pivotIndex + 1, high);
}

int main() {

    int n;
    cin >> n;
```

```
    vector<int> arr(n);

    for(int i = 0; i < n; i++)
        cin >> arr[i];

    quickSort(arr, 0, n - 1);

    for(auto x : arr)
        cout << x << " ";

    return 0;
}
class Solution {
public:

    vector<int> bfs(vector<vector<int>>& adj) {

        int n = adj.size();

        queue<int> q;

        vector<bool> vis(n, false);

        vector<int> output;

        q.push(0);

        vis[0] = true;

        while(!q.empty()) {

            int node = q.front();
```

```
            q.pop();

            output.push_back(node);

            for(auto adjnode : adj[node]) {

                if(!vis[adjnode]) {

                    vis[adjnode] = true;

                    q.push(adjnode);
                }
            }
        }

        return output;
    }
};
class Solution {
public:

    void dfs_solver(int src,
                    vector<vector<int>>& adj,
                    vector<bool>& vis,
                    vector<int>& output) {

        vis[src] = true;

        output.push_back(src);

        for(auto adjnode : adj[src]) {
```

```
            if(!vis[adjnode]) {

                dfs_solver(adjnode, adj, vis, output);
            }
        }
    }

    vector<int> dfs(vector<vector<int>>& adj) {

        int n = adj.size();

        vector<int> output;

        vector<bool> vis(n, false);

        dfs_solver(0, adj, vis, output);

        return output;
    }
};
```